

Baseline Reprodutível: Primeiro Sistema e Pipeline Executável

João Pedro Tentis¹, Pedro Mion Braga¹

¹PESC – Universidade Federal do Rio de Janeiro (UFRJ)

1. Links

- GitHub: <https://github.com/jtentis/projeto-BMT>
- Overleaf: <https://tinyurl.com/56nx8f4c>

2. Descrição do baseline

O baseline implementado nesta entrega tem como objetivo oferecer uma referência mínima, executável e reproduzível para o problema de auditoria automática de trabalhos acadêmicos. Nesta versão inicial, a abordagem foi simplificada para privilegiar a estabilidade do pipeline, facilidade de reprodução e uma primeira linha de comparação para as próximas iterações.

A escolha foi por um baseline heurístico e totalmente offline. Em vez de treinar um classificador DoCO completo nesta etapa, o sistema identifica seções do artigo a partir de títulos numerados e compara os trechos de *Introdução* com os trechos de entrega disponíveis, priorizando *Resultados* e, na ausência deles, *Conclusão*. A comparação textual é feita com vetorização TF-IDF e similaridade do cosseno.

Essa decisão reduz a dependência de dados anotados, evita o download de modelos e deixa o processo facilmente repetível em qualquer máquina com Python e as dependências definidas.

3. Pipeline completo

O pipeline executado pelo baseline segue as etapas abaixo:

1. **Ingestão de dados:** leitura de todos os PDFs em `data/raw/`.
2. **Extração de texto:** conversão do PDF em texto com a biblioteca `pypdf`.
3. **Pré-processamento:** remoção de hifenização artificial, normalização de espaços e normalização textual para busca de seções.
4. **Segmentação heurística:** identificação de cabeçalhos como 1. *Introdução*, *Resultados* e 6. *Conclusão*, com registro explícito de seções ausentes.
5. **Indexação e inferência:** vetorização dos trechos com TF-IDF e cálculo da similaridade do cosseno entre a seção de promessa e a melhor seção de entrega disponível.
6. **Exportação de saída:** geração de artefatos intermediários e arquivos finais com métricas e detalhes da comparação.

4. Ambiente e dependências

- Linguagem: Python 3.13
- Bibliotecas principais:
 - pypdf==5.4.0
 - scikit-learn==1.6.1
- Instalação:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

5. Como reproduzir

Comando exato para rodar o baseline:

```
python -m src.run_baseline --input_dir data/raw --
output_dir reports/baseline
```

Arquivos gerados como saída:

- reports/baseline/extraction_summary.json
- reports/baseline/extracted_text/*.txt
- reports/baseline/segmentation/*_sections.json
- reports/baseline/alignment/results.json
- reports/baseline/metrics.csv

6. Primeiros resultados

O corpus atual versionado no repositório contém alguns PDFs de exemplo (extraídos do Pantheon UFRJ) mas, as métricas abaixo são referentes ao documento `data/raw/proposta_bmt.pdf`. Para esse documento, o baseline detectou *Introdução* e *Conclusão*, não encontrou uma seção de *Resultados* com o padrão heurístico adotado e comparou a introdução com a conclusão.

Documento	Status	Promessa	Entrega	Score	Rótulo
proposta_bmt	ok	introduction	conclusion	0.0640	low

Table 1. Métricas iniciais do baseline.

O comportamento observado é coerente com o tipo de documento processado. Como o PDF usado neste baseline é uma proposta de projeto, ele não contém uma seção de resultados consolidada e, por isso, a comparação foi feita contra a conclusão. O score baixo era esperado e serve como referência inicial para as próximas iterações.

7. Limitações

- O baseline depende de cabeçalhos simples e numerados para segmentar seções.
- Ainda não há classificador DoCO treinado.
- O corpus inicial é pequeno e tem função principalmente demonstrativa.
- TF-IDF oferece uma comparação lexical útil como ponto de partida, mas limitada semanticamente.